

Übung: Perspektivische Bildersetzung mit OpenCV

Übung für Lernpunkte

Mit dem Lösen dieser Übung könnt ihr zwei von insgesamt sechs Lernpunkten erarbeiten.

Ausgangslage

Ein Bild hängt schräg an einer Wand. Du willst den Inhalt des Rahmens durch ein eigenes Bild ersetzen – perspektivisch korrekt eingepasst, wie es AR-Apps, Sportticker-Einblendungen oder virtuelle Werbebanner tun können. Dafür müssen wir das Bild perspektivisch korrekt einpassen.

Ziele der Übung

- Bilder in OpenCV laden können
- Rechnen mit Bildkoordinaten
- Geometrische Transformationen anwenden

Aufgabe

1. Nehmt ein Bild einer Plakatwand, eines Bildes an einer Wand o.ä., entweder selbst aufgenommen oder z.B. aus dem Internet. Dabei soll die Plakatwand nicht frontal aufgenommen sein, sondern wie im untenstehenden Beispiel perspektivisch verzerrt sein.
2. Wählt ein Bild aus, das ihr in diese Plakatwand einpassen möchtet. Achtet darauf, dass das Seitenverhältnis ungefähr übereinstimmt.
3. Berechnet die Pixelkoordinaten der Plakatwand-Ränder (hilfreiche OpenCV-Funktionen sind weiter unten aufgelistet).
4. Überlegt euch, welche Pixelkoordinaten das einzupassende Bild hat
5. Berechnet die nötige Transformation des einzupassenden Bildes
6. Wendet die Transformation an. Ihr müsst dabei nicht zwingend wie im Beispiel unten gezeigt die Überlagerung visualisieren, das entsprechend «verzerrte» Bild genügt.

Beispielbilder



OpenCV – nützliche Hinweise

Installation in einem Python Environment

```
pip install opencv-python
```

Verwenden

```
import cv2
```

Bild laden

```
cv2.imread('<pfad>')
```

Bild speichern

```
cv2.imwrite('<pfad>')
```

Bildkoordinaten durch Klicken erhalten, z.B. (Code auch [hier](#) auf GitHub)

```
import cv2 import numpy as np

BILD_Pfad = 'szene.png' LABELS = ['oben-links', 'oben-rechts', 'unten-rechts', 'unten-links']
FARBEN = [(60, 60, 220), (50, 200, 50), (220, 130, 30), (30, 160, 220)]

bild = cv2.imread(BILD_Pfad)
anzeige = bild.copy()
geklickte_punkte = []

def on_click(event, x, y, flags, param):
    if event != cv2.EVENT_LBUTTONDOWN:
        return if len(geklickte_punkte) >= 4:
        return

    idx = len(geklickte_punkte)
    geklickte_punkte.append([x, y])

    cv2.circle(anzeige, (x, y), 8, FARBEN[idx], -1)
    cv2.putText(anzeige, f'{{LABELS[idx]}} ({{x}},{{y}})',
                (x + 10, y - 10), cv2.FONT_HERSHEY_SIMPLEX,
                0.55, FARBEN[idx], 2)

    if len(geklickte_punkte) == 4:
        pts = np.array(geklickte_punkte, dtype=np.int32)
        cv2.polylines(anzeige, [pts], isClosed=True,
                      color=(0, 220, 220), thickness=2)

    cv2.imshow('Eckpunkte setzen', anzeige)

    if len(geklickte_punkte) == 4:
        print('\ndst_pts = np.float32([')
        for pt in geklickte_punkte:
            print(f'    {pt},')
        print('])')
        print('\nFenster schliessen mit beliebiger Taste.')

cv2.namedWindow('Eckpunkte setzen')
cv2.setMouseCallback('Eckpunkte setzen', on_click)
cv2.imshow('Eckpunkte setzen', anzeige)
cv2.waitKey(0) # "q" drücken um das Fenster zu schliessen
cv2.destroyAllWindows()
```

Perspektiven-Koordinatentransformation berechnen

```
cv2.getPerspectiveTransform(src_pts, dest_pts)
```

Perspektiven-Transformation anwenden

```
cv2.warpPerspective
```

Abgabe (auf Moodle) als zip-Datei, bis am 29.5.2026

- Python Code (Code-Qualität o.ä. wird nicht bewertet)
- Verwendete Bilder
- Output als «verzerrtes» Bild und Transformationsmatrix
- Kein Begleittext/Bericht o.ä. nötig